

Лабораторная работа №4

Вариант: № 10

Тема: Модули, исключения, файлы и сериализация данных в Python

Цель работы: Получить практические навыки разработки многофайловых Python-приложений, обработки исключительных ситуаций и сохранения данных во внешних файлах.

Постановка задачи:

Необходимо доработать объектную модель из лабораторной работы №3 или реализовать новую предметную область из индивидуального задания.

Программа должна:

1. использовать несколько Python-модулей;
2. содержать классы предметной области;
3. обрабатывать некорректный пользовательский ввод;
4. сохранять данные во внешний файл;
5. загружать данные при запуске;
6. преобразовывать объекты в JSON;
7. восстанавливать объекты из JSON;
8. предоставлять консольное меню.

Краткие теоретические сведения:

- Разделение кода на модули и пакеты. По мере роста проекта код делят на отдельные файлы (модули, .py) и папки-пакеты (с `__init__.py`). Так проще поддерживать проект, тестировать части отдельно и следовать принципу разделения ответственности (например: `models.py` — модели, `services.py` — логика, `storage.py` — работа с файлами). Импортировать можно весь модуль или отдельные объекты, также используют псевдонимы (`as`).
- Обработка ошибок и исключений. Ошибки в Python — это исключения (`ValueError`, `IndexError`, `KeyError` и др.). Их обрабатывают через `try/except`, блок `else` срабатывает, если ошибок не было, а `finally` — всегда (часто для освобождения ресурсов). Можно вручную генерировать исключения через `raise` и создавать свои типы исключений, наследуясь от `Exception`.

- Работа с файлами. Основной способ — через контекстный менеджер with open(...) as file: он гарантирует закрытие файла. Режимы доступа: чтение (r), запись с очисткой (w), добавление (a), создание нового (x).
- Форматы данных: CSV и JSON. Для CSV используют модуль csv (в том числе DictReader для чтения в виде словарей). JSON нужен для обмена структурированными данными и работы с API. Модуль json предоставляет dumps/dump (сериализация) и load/loads (десериализация). Параметр indent делает JSON читаемым.
- Сериализация объектов. Стандартные объекты классов нельзя напрямую сохранить в JSON: сначала их преобразуют в словари, а при чтении — обратно создают экземпляры класса.
- Архитектура. Важно разделять хранение данных, бизнес-логику и модели: main.py не должен напрямую работать с файлами — эту задачу делегируют модулю хранилища.

Исходный код программы:

main.py

```
import models
import storage
import exceptions
import services

#---чтение из джсон
dish_list_json = storage.read_dish_json()
dish_list_class = []
orders_list_class = []

#---создание списка объектов класса диш
try:
    for i in dish_list_json:
        dish = models.Dish(i[0], i[1], i[2], i[3])
        dish_list_class.append(dish)

    while True:
        print('Чё хочешь',
              '1 - добавить блюдо',
              '2 - удалить блюдо',
              '3 - изменить цену',
              '4 - поиск по имени и/или категории',
              '5 - создать заказ',
              '6 - выход', sep='\n')
        menu = input()
        if menu == '1':
            print("Добавление Блюда")
            dname = input("Введите название блюда ")
            dcategory = input("Введите категорию ")
            dprice = float(input("Введите цену "))
            dcaloric = input("Введите количество калорий ")
```

```

        newdish = models.Dish(dname, dcategory, dprice, dcaloric)
        dish_list_class.append(newdish)
        storage.write_dish_json(newdish)

    elif menu == '2':
        dname = input('Введите название блюда, которое хотите
удалить: ')
        storage.delete_dish_by_name(dname)

    elif menu == '3':
        dname = input('Введите название блюда: ')
        dprice = float(input('Введите новую цену: '))
        storage.update_dish_price_by_name(dname, dprice)

    elif menu == '4':
        dname = input('Введите название блюда: ')
        print(storage.find_dishes(dname))

    elif menu == '5':
        while True:
            customer_name = input('Введите имя заказчика ')
            order =
models.Order(services.new_order(dish_list_class), customer_name)
            order.price = services.final_price(order.names,
dish_list_class)
            print(f'Ваш заказ: {order.names}',
                  f'Стоимость: {order.price}',
                  '1 - подтвердить, 2 - отказаться и повторить
попытку, 3 - отказаться и вернуться в меню', sep='\n')
            menu2 = input()
            if menu2 == '1':
                order.order_done()
                orders_list_class.append(order)
                break
            elif menu2 == '2':
                continue
            elif menu2 == '3':
                break

    elif menu == '6':
        print('Завершение работы программы')
        break

    else:
        print('Некорректный ввод, повторите попытку')

except exceptions.FuckingLessZeroPrice:
    print('Цена блюда должна быть больше нуля!')

except ValueError:
    print("Некорректные данные, попробуйте ещё раз: ")

```

models.py

```
import exceptions
```

```

class Dish:
    def __init__(self, name, category, price, caloric):
        self.__name = name
        self.__category = category
        self.price = price
        self.__caloric = caloric

    @property
    def name(self):
        return self.__name

    @property
    def category(self):
        return self.__category

    @property
    def caloric(self):
        return self.__caloric

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, new_price):
        if new_price < 0:
            raise exceptions.FuckingLessZeroPrice()
        self.__price = new_price

class Order:
    def __init__(self, names, customer_name):
        self.__names = names
        self.__customer_name = customer_name
        self.__price = 0
        self.status = False

    @property
    def price(self):
        return self.__price

    @property
    def names(self):
        return self.__names

    @price.setter
    def price(self, new_price):
        if new_price < 0:
            raise exceptions.FuckingLessZeroPrice()
        else:
            self.__price = new_price

    def order_done(self):
        print('Заказ успешно оформлен!')
        self.status = True

```

services.py

```

def final_price(names, data):
    price = 0
    for line in data:
        for name in names:
            if name == line.name:
                price += float(line.price)
    return price

def new_order(data):
    orders = []
    while True:
        order = input('Введите название позиции, или 0 если хотите закончить ')
        if order == '0':
            break
        else:
            for line in data:
                print(order, line, line.name)
                if order == line.name:
                    orders.append(order)
                    break
            else:
                print('Такой позиции нет в меню')

    return orders

```

storage.py

```

import os
import json
import csv
from pydantic import BaseModel

class wDish(BaseModel):
    name: str
    category: str
    price: float
    caloric: float

def sync_to_json(dishes: list[wDish]):
    file_path = 'dish.json'
    # Дампим весь список моделей в валидный JSON-массив
    data = [dish.model_dump() for dish in dishes]
    with open(file_path, 'w', encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=4)

def append_to_csv(dish: wDish):
    file_path = 'dish.csv'
    file_exists = os.path.exists(file_path)

    with open(file_path, 'a', newline='', encoding="utf-8-sig") as f:

```

```

        writer = csv.writer(f, delimiter=';')
        # Если файл создается впервые, пишем заголовки
        if not file_exists:
            writer.writerow(['name', 'category', 'price', 'caloric'])
        writer.writerow([dish.name, dish.category, dish.price,
dish.caloric])

def rewrite_csv(dishes: list[wDish]):
    file_path = 'dish.csv'
    with open(file_path, 'w', newline='', encoding="utf-8-sig") as f:
        writer = csv.writer(f, delimiter=';')
        writer.writerow(['name', 'category', 'price', 'caloric'])
        for dish in dishes:
            writer.writerow([dish.name, dish.category, dish.price,
dish.caloric])

def load_from_json() -> list[wDish]:
    file_path = 'dish.json'
    if not os.path.exists(file_path):
        return []
    with open(file_path, 'r', encoding="utf-8") as f:
        try:
            data = json.load(f)
            return [wDish(**item) for item in data]
        except json.JSONDecodeError:
            return []

def write_dish_json(list_):
    dishes = load_from_json()

    # Проверка на дубликат имени
    for existing_dish in dishes:
        if existing_dish.name.lower() == list_.name.lower():
            print(f"Ошибка: Блюдо с именем '{list_.name}' уже
существует в базе!")
            return

    dish = wDish(name=list_.name, category=list_.category,
price=list_.price, caloric=list_.caloric)
    dishes.append(dish)
    sync_to_json(dishes)
    append_to_csv(dish)

    print(f"Блюдо '{list_.name}' успешно добавлено.")

def read_dish_json():
    dishes = load_from_json()
    list_ = []

```

```

for dish in dishes:
    list_2 = []
    for i in dish:
        list_2.append(i[1])
    list_.append(list_2)
return list_

def update_dish_price_by_name(target_name: str, new_price: float):
    dishes = load_from_json()
    updated = False

    for dish in dishes:
        if dish.name == target_name:
            dish.price = new_price
            updated = True

    if updated:
        sync_to_json(dishes)
        rewrite_csv(dishes)
        print(f"Блюдо '{target_name}' успешно обновлено.")
    else:
        print(f"Блюдо '{target_name}' не найдено.")

def delete_dish_by_name(target_name: str):
    dishes = load_from_json()
    initial_count = len(dishes)
    dishes = [dish for dish in dishes if dish.name != target_name]

    if len(dishes) < initial_count:
        sync_to_json(dishes)
        rewrite_csv(dishes)
        print(f"Блюдо '{target_name}' успешно удалено.")
    else:
        print(f"Блюдо '{target_name}' не найдено.")

def find_dishes(search_query):
    dishes = load_from_json()
    results = []
    for dish in dishes:
        if search_query.lower() in dish.name.lower():
            results.append(dish)
    return results

```

exceptions.py

```

class FuckingLessZeroPrice(Exception):
    pass

```